SE/COM S 3190
Fall 2026



In-Class Activity 10

AGENDA

Fakestore Catalog API

Expected Final Output

Important JS Functions

- map

- filter

TailwindCSS

Part – 1



Fakestore Catalog API

Fakestore Catalog API

https://fakestoreapi.reactbd.com

Click

ABOUT OUR MOCK API

Fake Store API

Fake Store API is a free, reliable REST API designed specifically for frontend developers, UX/UI designers, and students to streamline e-commerce prototyping. Stop wasting time building backend services from scratch and start fetching realistic JSON data instantly.

- Access 200+ products, 500+ orders, and detailed user profiles.
- Test shopping carts, categories, wishlists, and payments.
- Perfect for tutorials, live demos, and rapid MVP development.
- **Zero authentication required**—start instantly!

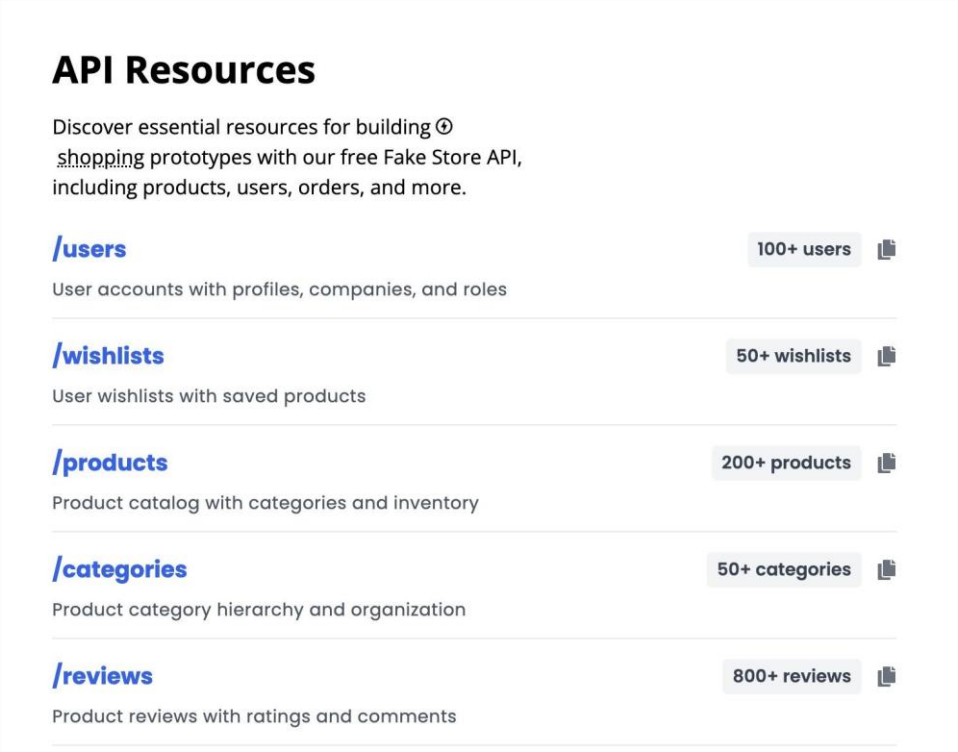
Get Source Code Read API Docs



Resources

https://fakestoreapi.reactbd.com

Click



Data – 20 Products




```
[
      {
            "_id": 1,
            "title": "Long sleeve Jacket",
            "isNew": true,
            "oldPrice": "200",
            "price": 150,
            "discountedPrice": 135,
            "description": "Lorem ipsum dolor sit amet, consectetur adipisicing elit. Nulla non magni
      facili blanditiis molestias soluta eveniet illum accusantium eius mollitia eligendi, ex iste
      doloribus magnam.",
            "category": "women",
            "type": "jacket",
            "stock": 50,
            "brand": "FashionTrend",
            "size": [
              "S",
              "M",
              "L"
            ],
            "image":  "https://images.pexels.com/photos/2584269/pexels-photo-2584269.jpeg",
            "rating": 4
         },
```

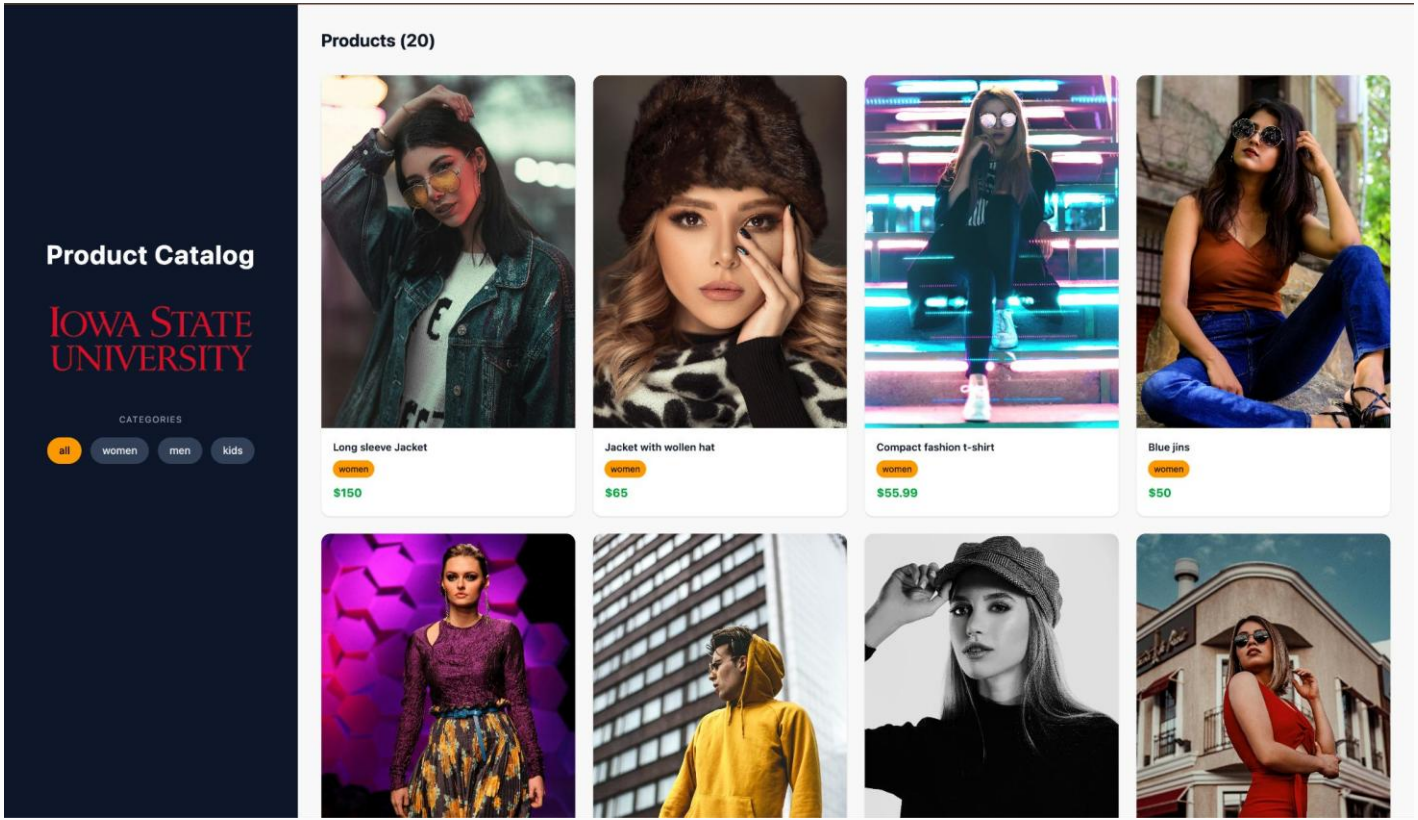

... to continue

Categories

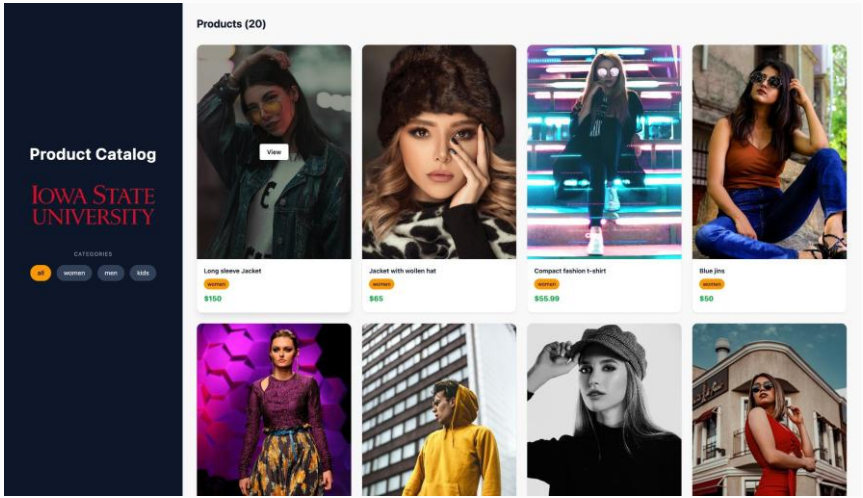
- women
- men
- kids

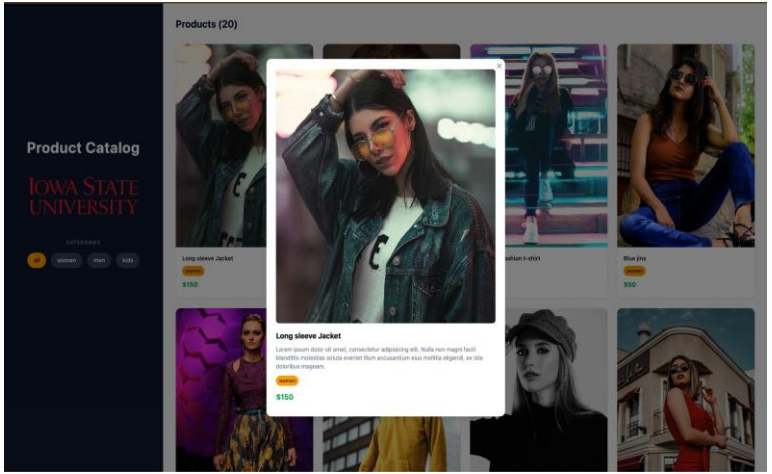
Part 1 Output

Expected Part 1 Output

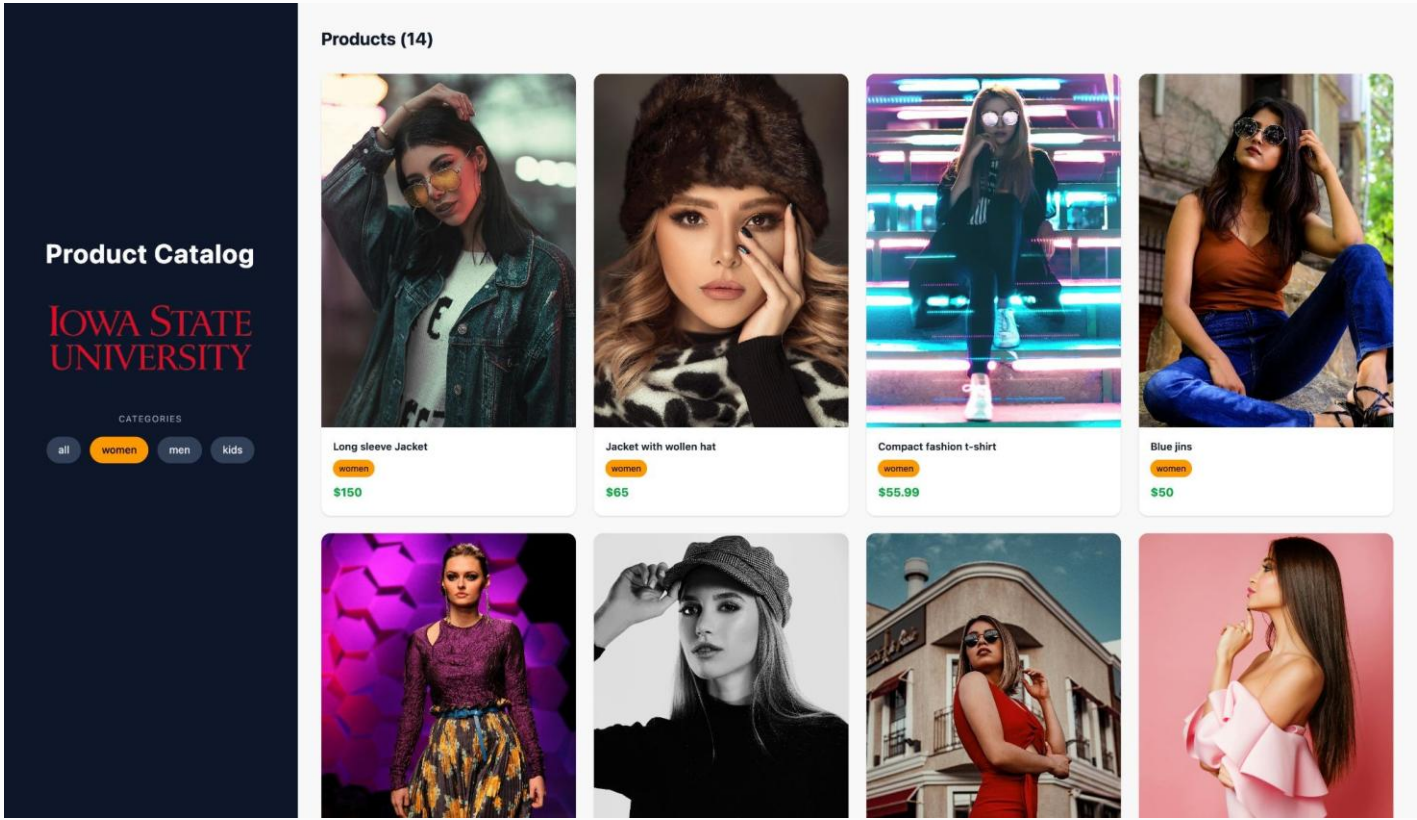


Expected Part 1 Output

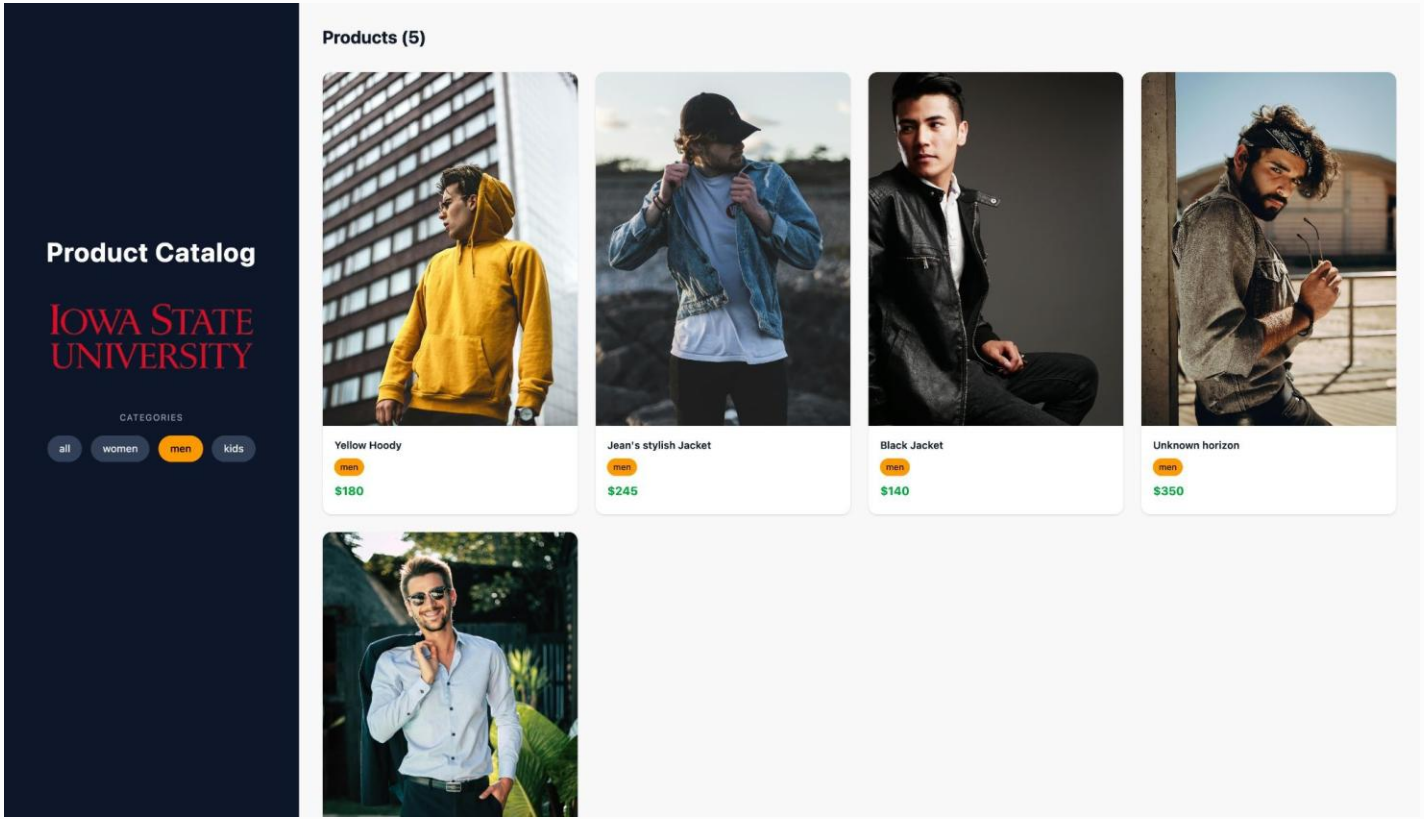




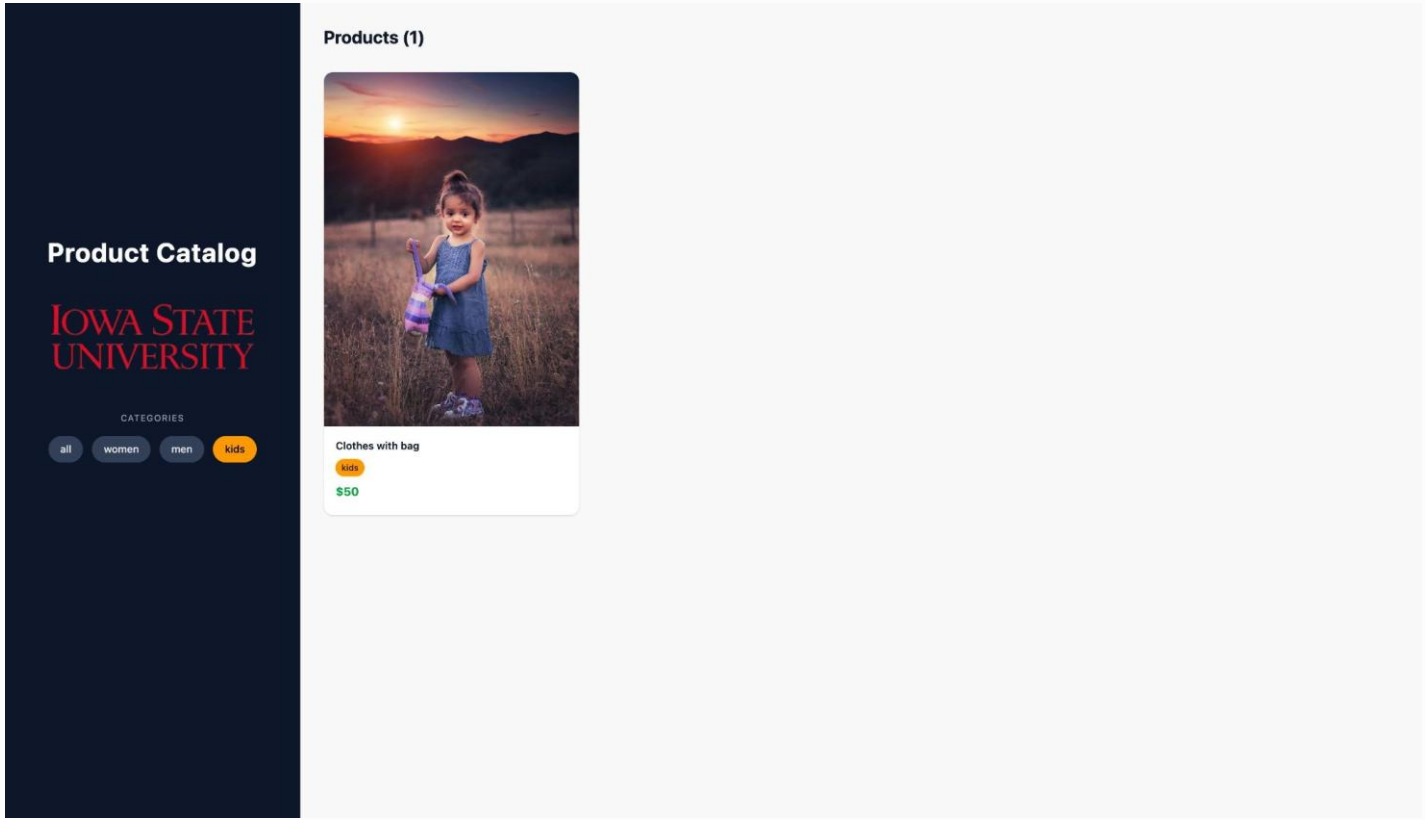
Expected Part 1 Output



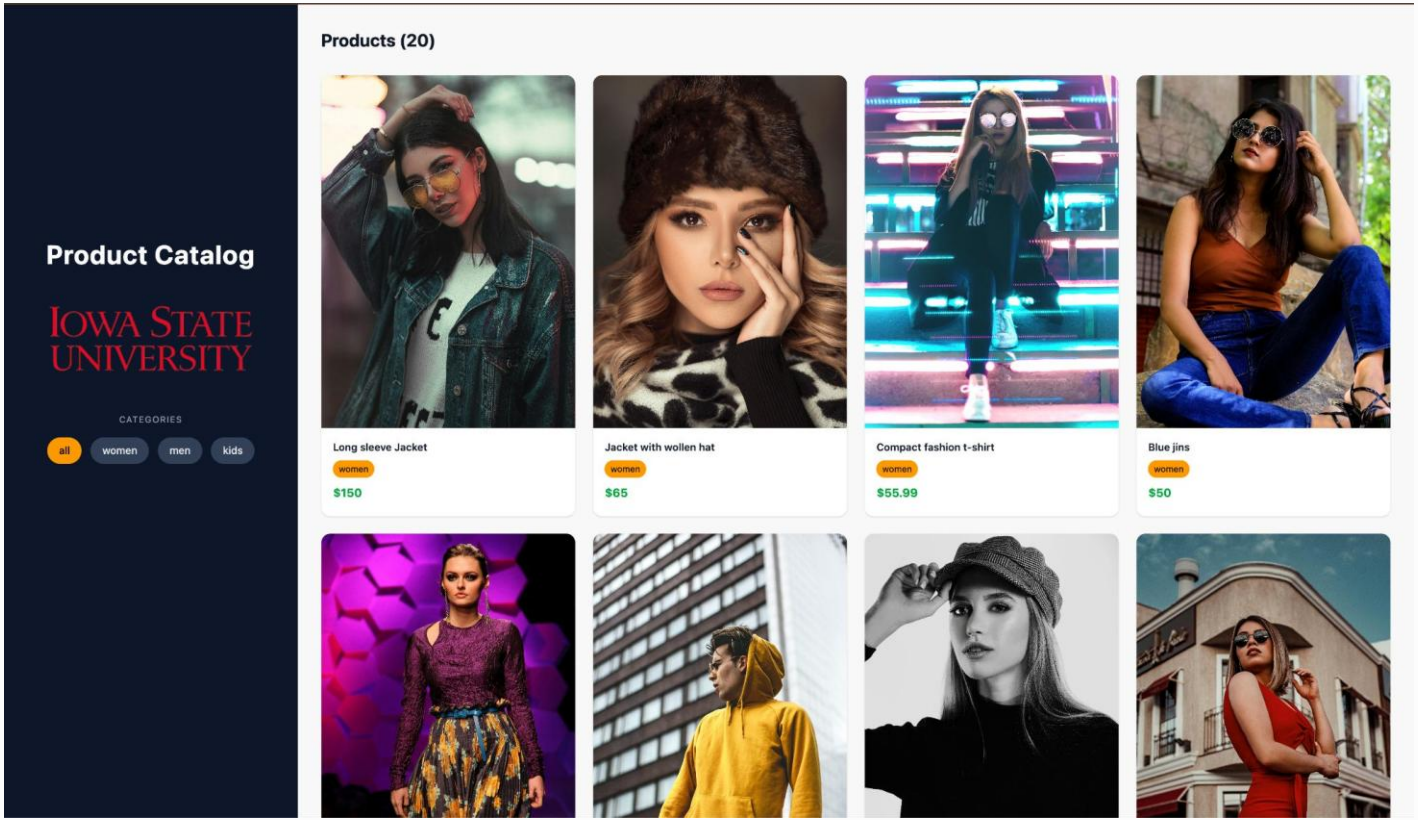
Expected Part 1 Output



Expected Part 1 Output

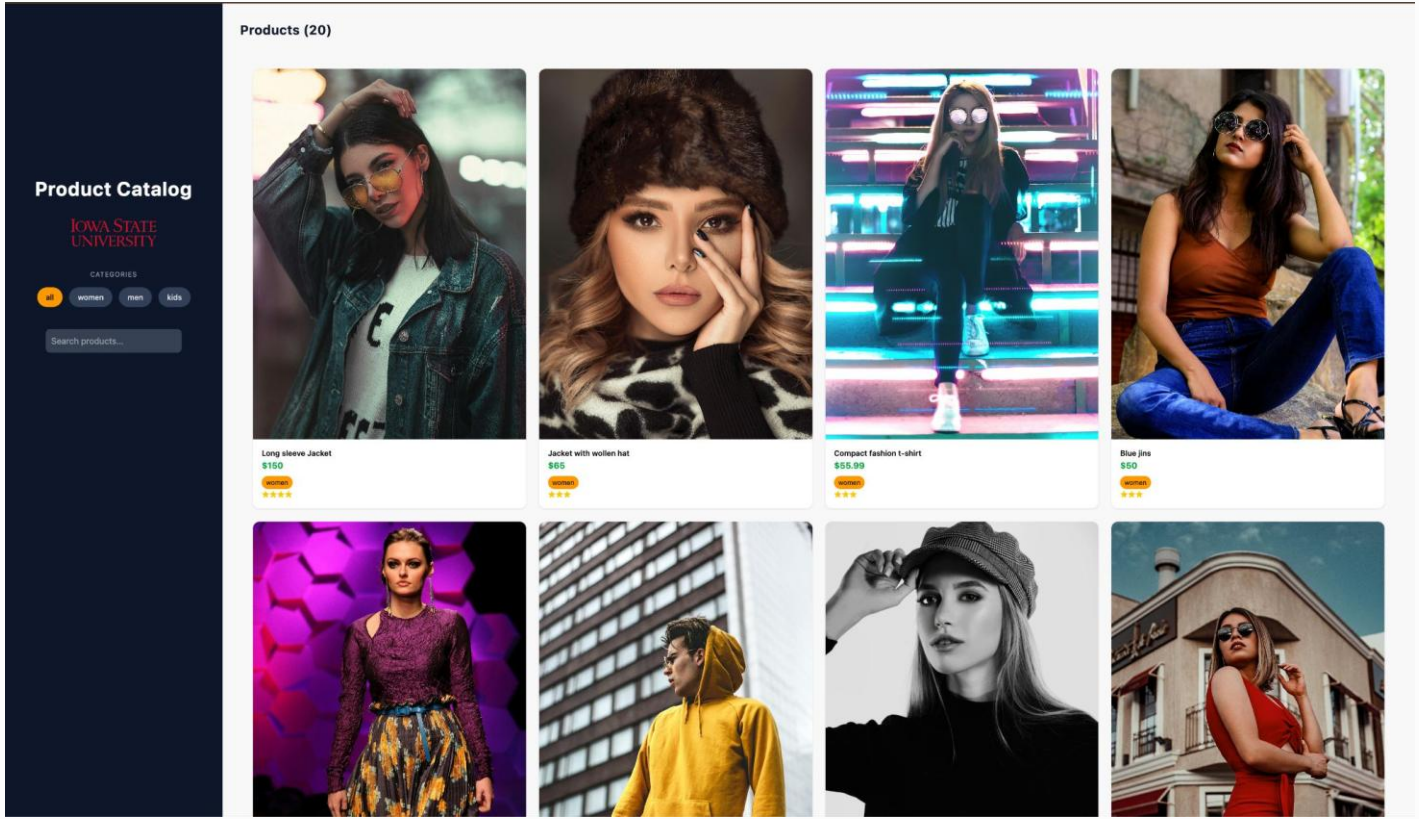


Expected Part 1 Output

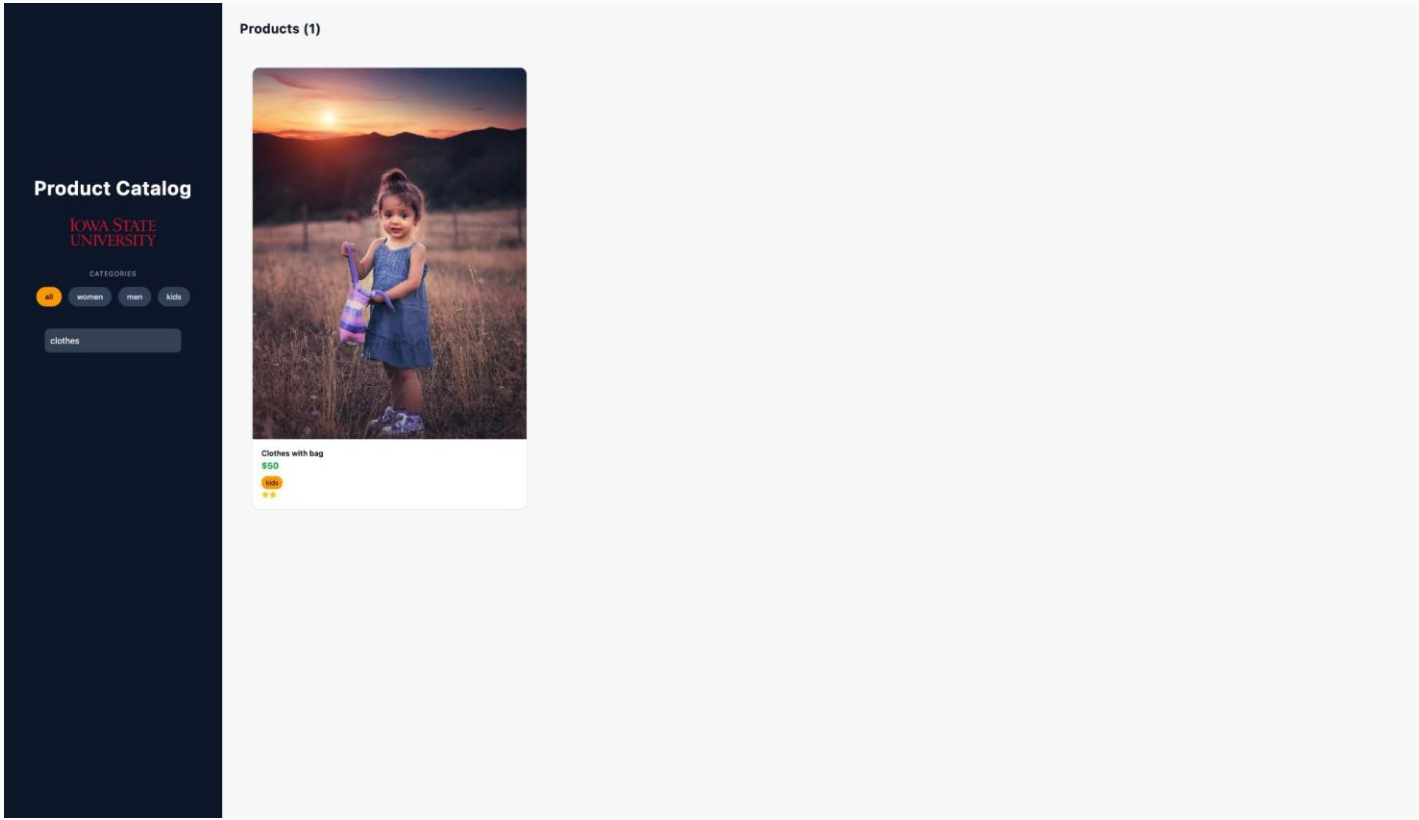


Part 2 Output

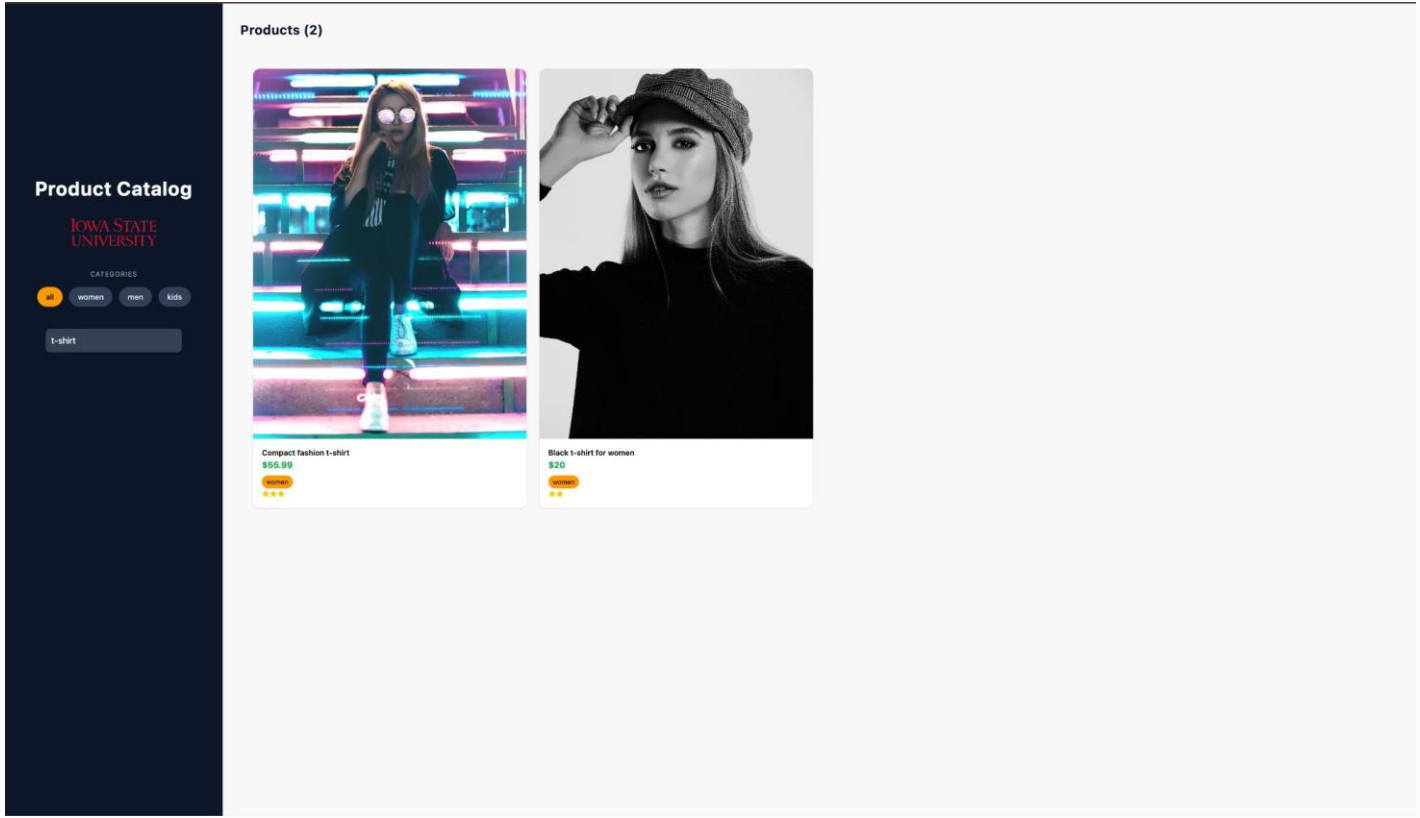
Expected Part 1 + Part 2 Output



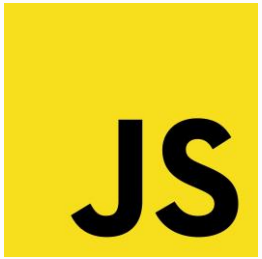
Expected Part 1 + Part 2 Output



Expected Part 1 + Part 2 Output



SE/COM S 3190
Fall 2026



Important JS Functions

map

Brief Introduction

Given an array:

```
const tas = [“Mahdi”, “Arafat”, “Phong”];
```

With the help of the map function, instead of having to write multiple lines of code to say Hi to every TA, we just write it once and map it to every element in the array like:

```
tas.map(function(ta) { return 'Hi ' + {ta});
```

Without & With `map` function

Without `map` function

```
> const tas = ["Mahdi", "Arafat", "Phong"];
< undefined
> console.log("Hi Mahdi");
  Hi Mahdi                                  VM252:1
< undefined
> console.log("Hi Arafat");
  Hi Arafat                                 VM265:1
< undefined
> console.log("Hi Phong");
  Hi Phong                                  VM279:1
< undefined
```

With `map` function

```
> tas.map((ta) => {
      console.log("Hi "+ta+"!");
  });
  Hi Mahdi!                                 VM617:2
  Hi Arafat!                                VM617:2
  Hi Phong!                                 VM617:2
< ▸ (3) [undefined, undefined, undefined]
```

filter

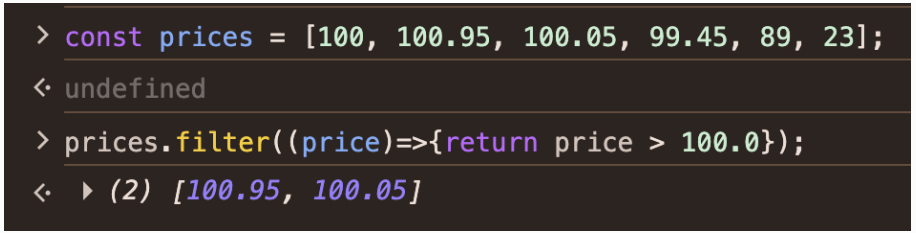
Brief Introduction

Given an array:

```
const prices = [100, 100.95, 100.05, 99.45, 89, 23];
```

As the name of the function suggests, it filters items from a given array

```
prices.filter((price)=>{return price > 100.0});
```





TailwindCSS

Setup TailwindCSS in your Project

Install

```
npm install tailwindcss
@tailwindcss/vite
```

Edit vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

// https://vite.dev/config/
export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

Edit index.css

```
@import "tailwindcss";

html,
body,
#root {
  height: 100%;
  width: 100%;
}
```

Create directories

src/components/

Create 2 files:

- Products.jsx
- Sidebar.jsx

src/data/

Download the fashion.js and categories.js files from Canvas and put them in this directory

Sidebar.jsx

Step #1 - Basic File Structure

```
const SideNavbar = ({ categories, active, setActive }) => {
  return (
    <div className="bg-[#0f172a] text-white min-h-screen p-6">
      {/* Content goes here */}
    </div>
  );
};
```

Step #2 - h1, img, p for the Sidebar

```
<h1 className="text-4xl font-bold tracking-wide mb-4 text-center">
  Product Catalog
</h1>

<img
  src={logo}
  alt="logo"
  className="w-40 mb-6 object-contain"
/>

<p className="text-xs uppercase tracking-widest text-gray-400 mb-4 text-center">
Categories
</p>
```

Step #3 - Add Category chips

```
<div className="flex flex-wrap justify-center gap-3">
  {categories.map((cat) => (
    <button
        key={cat}
        className="px-4 py-2 rounded-full text-sm font-medium bg-
        slate-700 text-gray-200"
    >
      {cat}
    </button>
  ))}
</div>
```

Step #4 - Set Active State & Conditional UI Rendering

To the button:

```
onClick={() => setActive(cat)}

className={`px-4 py-2 rounded-full text-sm font-medium transition
  ${
    active === cat
      ? "bg-amber-500 text-black shadow"
      : "bg-slate-700 text-gray-200 hover:bg-slate-600"
  }`}
```

Products.jsx

Step #1 - Basic File Structure

```
const Products = ({ products }) => {
  return (
  <div className="flex-1 p-8 bg-[#f8f8f8]">
    <div className="flex justify-between items-center mb-8">
      <h2 className="text-2xl font-bold text-gray-900">
         Products ({products.length})
      </h2>
    </div>
  </div>
  );
};
```

Step #2 - Add Grid Layout

```
<div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4
gap-6">
  {products.map((product, index) => (
    <div key={index}>
      {product.title}
    </div>
  ))}
</div>
```

Step #3 - Add Card Structure

```
<div className="bg-white rounded-xl shadow-sm p-4">
  <img src={product.image} alt="product" />
  <h3>{product.title}</h3>
  <p>${product.price}</p>
</div>
```

Step #4 - Add styling & layout

```
<div className="bg-white rounded-xl overflow-hidden shadow-sm hover:shadow-lg transition">
  <div className="h-120 overflow-hidden">
    <img
      src={product.image}
      className="w-full h-full object-cover"
    />
  </div>

  <div className="p-4">
    <h3 className="text-sm font-semibold">{product.title}</h3>
    <p className="text-green-600 font-bold">${product.price}</p>
  </div>
</div>
```

Step #5 - Add category & rating

```
<span className="inline-block mt-2 px-2 py-1 text-xs rounded-full bg-amber-500 text-
black">
  {product.category}
</span>

<div className="text-yellow-400 text-sm">
  {"★".repeat(Math.round(product.rating))}
</div>
```

Step #6 - Add hover overlay

```
<div className="relative h-120 overflow-hidden group">
  <img ... />

  <div className="absolute inset-0 bg-black/40 opacity-0 group-hover:opacity-100 flex
items-center justify-center">
    <button>View</button>
  </div>
</div>
```

Step #7 - Add states & connect to buttons

```
const [showModal, setShowModal] = useState(false);
const [selectedProduct, setSelectedProduct] = useState(null);

To the View Button:

className="bg-white px-4 py-2 rounded text-sm font-medium"
onClick={() => {
  setSelectedProduct(product);
  setShowModal(true);
}}
```

Step #8 - Add a modal to show the pop-up tab

```
{showModal && selectedProduct && (

        <div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50">
          {/* Modal Box */}
          <div className="bg-white p-6 rounded-xl w-[500px] relative shadow-lg">
            {/* Close Button */}
            <button
              className="absolute top-2 right-3 text-gray-500 hover:text-black"
              onClick={() => setShowModal(false)}
            >
              ×
            </button>

            {/* Image */}
            <div className="h-170 overflow-hidden rounded-lg">
              <img
                src={selectedProduct.image}
                alt="product"
                className="w-full h-full object-cover"
              />
            </div>

            {/* Info */}
            <div className="mt-4">
              <h2 className="text-lg font-bold">{selectedProduct.title}</h2>

              <p className="text-sm text-gray-500 mt-2">
                {selectedProduct.description}
              </p>
            </div>
          </div>
        </div>
      )}
```

App.jsx

Step #1 - Basic File Structure

```
const App = () => {
  return (
    <div>
      {/* Sidebar */}
      <Sidebar />

      {/* Products */}
      <ProductsComponent />
    </div>
  );
};
```

Step #2 - Add Layout

```
<div className="flex min-h-screen">
  <Sidebar />
  <ProductsComponent />
</div>
```

Step #3 - Add State & Pass to Props

```
const [active, setActive] = useState("all");

<Sidebar
  categories={Categories}
  active={active}
  setActive={setActive}
/>
```

Step #4 - Add filtering logic

```
const filteredProducts =
  active === "all"
    ? Products
    : Products.filter((p) => p.category === active);
```

Update the props for <ProductsComponent />

```
<ProductsComponent products={filteredProducts} />
```

Challenge (Optional)

Try this!

In the `fashion.js` file, each product includes additional fields such as `discountedPrice`, a `sizes` array, and the number of items available in stock.

For this challenge, make use of these fields by displaying the discounted price next to the original price, with the original price shown as striked out.

Also, include the available stock count for each product and clearly list the sizes that are available.

Part - 2

Continue editing

App.jsx

Step #0 - What you already have?

```
import React, { useState } from "react";
import SideNavbar from "./components/SideNavbar";
import ProductsComponent from "./components/Products";

import { Products } from "./data/fashion";
import { Categories } from "./data/categories";
import Sidebar from "./components/Sidebar";

const App = () => {
  const [active, setActive] = useState("all");

  // Filtering logic
  const filteredProducts =
    active === "all" ? Products : Products.filter((p) => p.category === active);

  return (
    <div className="flex min-h-screen">
      {/* Sidebar */}
      <Sidebar categories={Categories} active={active} setActive={setActive} />

      {/* Products Section */}
      <ProductsComponent products={filteredProducts} />
    </div>
  );
};

export default App;
```

Step #1 - Add another state variable

```
const [query, setQuery] = useState("");
```

Step #2 - Update the filtering logic to include this

The whole idea of updating the filtering logic is because we want the user to have an option to continue to have the option of filtering by categories while also searching for items by typing the name

```
const matchesSearch = p.title.toLowerCase().includes(query.toLowerCase());

return matchesCategory && matchesSearch;
```

Step #3 - Update the props in the Sidebar

Add these to the Sidebar component call so that the data entered from the component is editable

```
query={query}
setQuery={setQuery}
```

Continue editing

Sidebar.jsx

Step #1 - Update the prop variables in the component

```
const Sidebar = ({ categories, active, setActive, query, setQuery }) => {
```

Step #2 - Add a helper function

```
const handleChange = (e) => {
    setQuery(e.target.value);
};
```

Step #3 - Add a input handler

```
<div className="py-10">
   <input
      type="search"
      value={query}
      onChange={handleChange}
      placeholder="Search products..."
      className="w-[250px] p-2.5 rounded-lg bg-gray-700 text-white"
    />
</div>
```

Activity 10

Done